

SPARSE AUTOENCODERS REVEAL SELECTIVE REMAPPING OF VISUAL CONCEPTS DURING ADAPTATION

Hyesu Lim^{1,2*} **Jinho Choi**² **Jaegul Choo**² **Steffen Schneider**^{1,3†}

¹Institute of Computational Biology, Computational Health Center, Helmholtz Munich

²KAIST AI ³Munich Center for Machine Learning (MCML)

ICLR 2025

<https://arxiv.org/pdf/2412.05276>

<https://github.com/dynamical-inference/patchsae>

Motivation

- Foundation models like CLIP are adapted for downstream vision tasks.
- Adaptation often uses prompt-tuning methods (e.g., CoOp, CoCoOp, MaPLe), but **the underlying mechanism of adaptation remains unclear.**
- Open questions: What internal mechanisms change during this adaptation process?
 - *Does the model learn to perceive new visual concepts?*
 - *Or, does it learn to re-interpret what it already sees?*

Contributions

- Propose **PatchSAE**: a Sparse Autoencoder applied to patch-level features of CLIP's ViT.
- Extract interpretable concepts at granular levels (e.g., shape, color, semantics of an object) and their patch-wise spatial attributions
- Show that adaptation mostly remaps existing concepts rather than creating new ones.
- Provide a general interpretability framework compatible with adapted models like MaPLe.

MaPLE: Multi-modal Prompt Learning

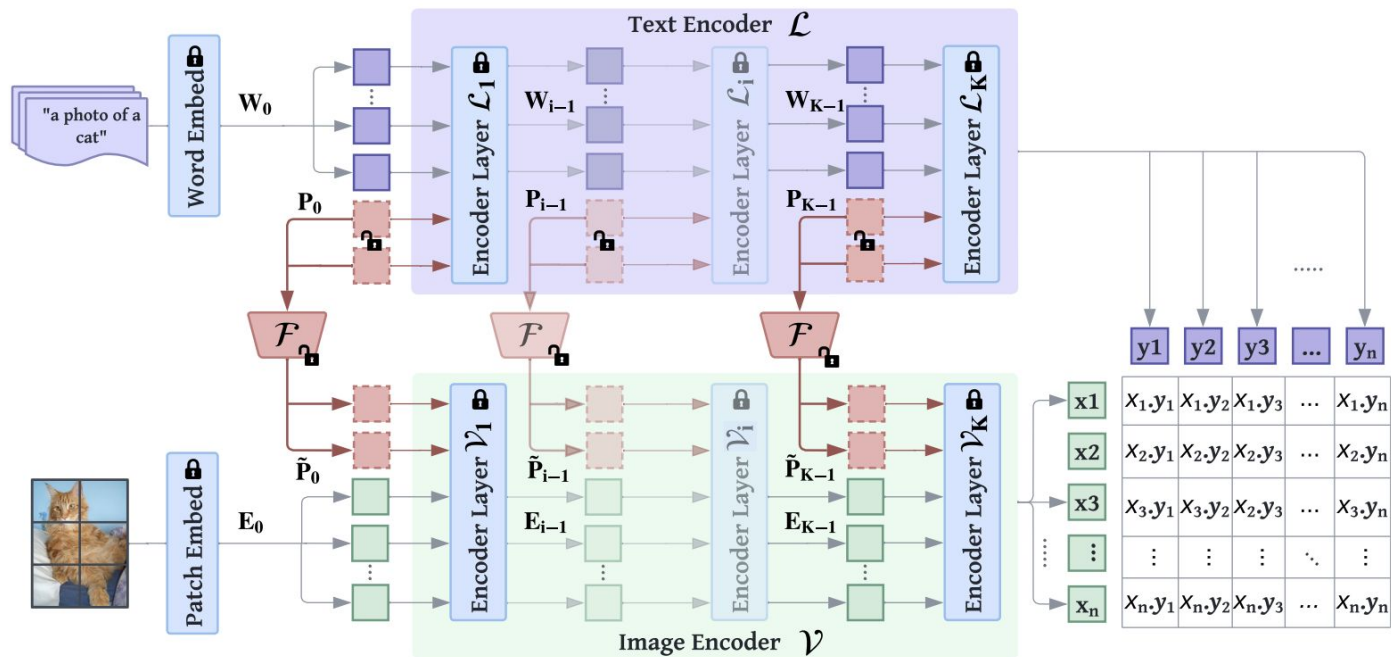


Figure 2. Overview of our proposed MaPLE (Multi-modal Prompt Learning) framework for prompt learning in V-L models. MaPLE tunes both vision and language branches where only the **context prompts** are learned, while the rest of the model is frozen. MaPLE conditions the vision prompts on language prompts via a V-L coupling function $\mathcal{F}$ to induce mutual synergy between the two modalities. Our framework uses deep contextual prompting where separate context prompts are learned across multiple transformer blocks.

Polysemanticity and SAEs

A challenge for Mechanistic Interpretability: **Polysemanticity**

- A single neuron can activate for multiple unrelated concepts
- This is thought to arise from **superposition**, where a model represents more features than it has dimensions

Sparse AutoEncoders (SAEs) is a tool to combat polysemanticity

- Decomposes a model's dense, polysemantic activation vector into a sparse, linear combination of **monosemantic** feature vectors (concepts)

PatchSAE: Model Architecture and Training Loss

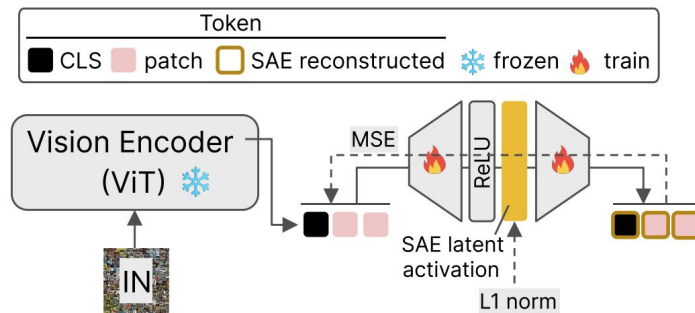
- A simple autoencoder
 - Encoder: Linear layer 🔥
 - Activation: ReLU
 - Decoder: Linear layer 🔥
- Input: Token embeddings from a CLIP ViT layer's residual stream.
- Key Idea: Applied to **CLS and all image tokens** from the residual stream output.

$$\mathbf{z} = \text{ViT}(\mathbf{x})[\text{hook layer}]$$

$$\text{SAE}(\mathbf{z}) = (g \circ \phi \circ f)(\mathbf{z}) = W_D^\top \phi(W_E^\top \mathbf{z})$$

$$\mathcal{L}_{\text{SAE}} = \|\text{SAE}(\mathbf{z}) - \mathbf{z}\|_2^2 + \lambda_{l_1} \|\phi(f(\mathbf{z}))\|_1.$$

- **Reconstruction (MSE)**: Ensures the original representation can be recovered
- **Sparsity (L1)**: Encourages only a few latents to be active for any given input.



(a) PatchSAE on CLIP ViT

PatchSAE: Training Details

Use a CLIP with ViT-B/16

- Input: 14×14 image tokens + 1 CLS token
- Model: 12 attention layers, with model dimension $d_{\text{ViT}} = 768$

PatchSAE configuration:

- Expansion factor to 64 -> SAE latent dimension d_{SAE} of 49,152
- Input to SAE: ViT output from the residual stream of the **11th layer**

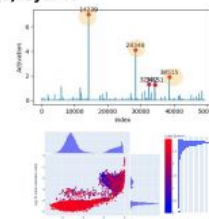
Training details:

- SAE input/ output shape: (number of samples, token length, model dimension d_{ViT})
- Loss: Mean Squared Error (MSE), averaged over all tokens.
- Regularization: L1 sparsity on SAE latent activations.

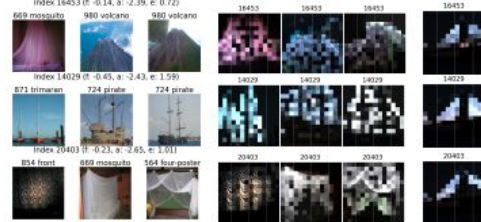
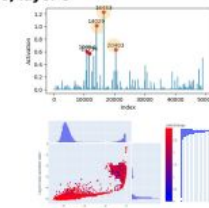
Why is the 11th layer?

- SAE latents on the deeper layers provide semantically richer information
- There is some loss of 'pure' visual information in the last (12th) layer.

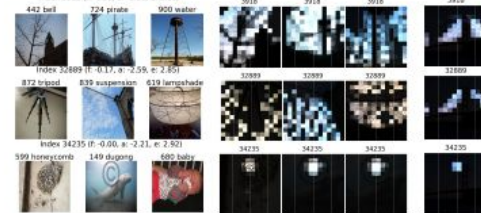
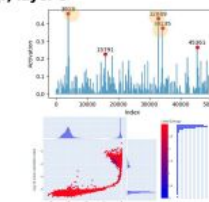
(d) layer 11



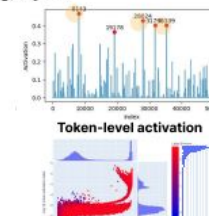
(e) layer 8



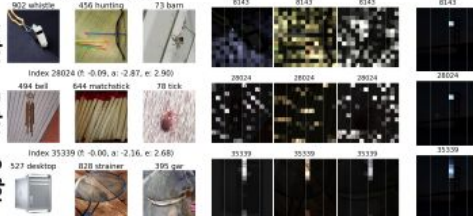
(f) layer 5



(g) layer 2



Top-1
Top-2
Top-3



Statistics of SAE_layer i

Reference images

Segmentation mask
of reference images

Segmentation mask
of input image

PatchSAE: How to Interpret the Learned Concepts?

Method 1: Reference images for SAE latents (Concept Discovery)

How do we “name” a latent direction?

- For each latent, find the top-k images from a dataset (e.g., ImageNet) that activate it most strongly.
- The common visual element in these images defines the concept.
 - e.g., If a latent is highly active for images of cats, label it the “cat concept”.

PatchSAE: How to Interpret the Learned Concepts?

Method 2: Summary statistics of SAE latents (Concept Characterization)

What are the properties of a concept?

- **Activated Frequency:** Is it a common or rare concept?
- **Mean Activation Value:** How "confident" is the model about this concept?
- **Label Entropy:** Is the specific to one class (low entropy) or general across many classes (high entropy)?

$$\text{prob}_c = \frac{\text{sum}_c}{\sum_{c \in C} \text{sum}_c}, \quad \text{entropy} = - \sum_{c \in C} (\text{prob}_c \log \text{prob}_c),$$

PatchSAE: How to Interpret the Learned Concepts?

Method 3: Discovering active SAE latents in diverse levels

- **Patch-level activation:**
 - The activation of an SAE latent for a specific patch indicates the presence of that latent's concept (e.g., a "dog's nose" concept activating for a patch containing a dog's nose).
- **Image-level activation:** To get a global understanding for an entire image, patch-level activations are aggregated.
 - The continuous SAE latent activation value for each patch ($h_{i,j}[s]$) is binarized by a threshold (τ) to determine if it is "active" (1) or "inactive" (0).
 - The number of active patches for a specific latent (s) is counted across the entire image (i). This count becomes the image-level activation ($a_i[s]$).

PatchSAE: How to Interpret the Learned Concepts?

Method 3: Discovering active SAE latents in diverse levels

- **Class-level and dataset-level activation:**
 - Class-Level Activation ($a_c[s]$): The image-level activations are summed up for all images belonging to the same class (c).
 - Dataset-Level Activation ($a_D[s]$): The image-level activations are summed across all images in a dataset (D).

PatchSAE: Analysis Methods & Evaluation Setup

Method 4: Localizing patch-level SAE latent activations

PatchSAE enables localizing which parts of an image activate a specific SAE latent concept.

- Method:
 - Treat patch-level SAE activations as a soft segmentation mask.
 - For a given image $\mathbf{x}_i$ and latent index s , multiply each patch $\mathbf{x}_{i,j}$ by its corresponding activation value $h_{i,j}[s]$.
- Visualization: Highlights patches where a specific concept (latent) is active.

Analyzing SAE latents

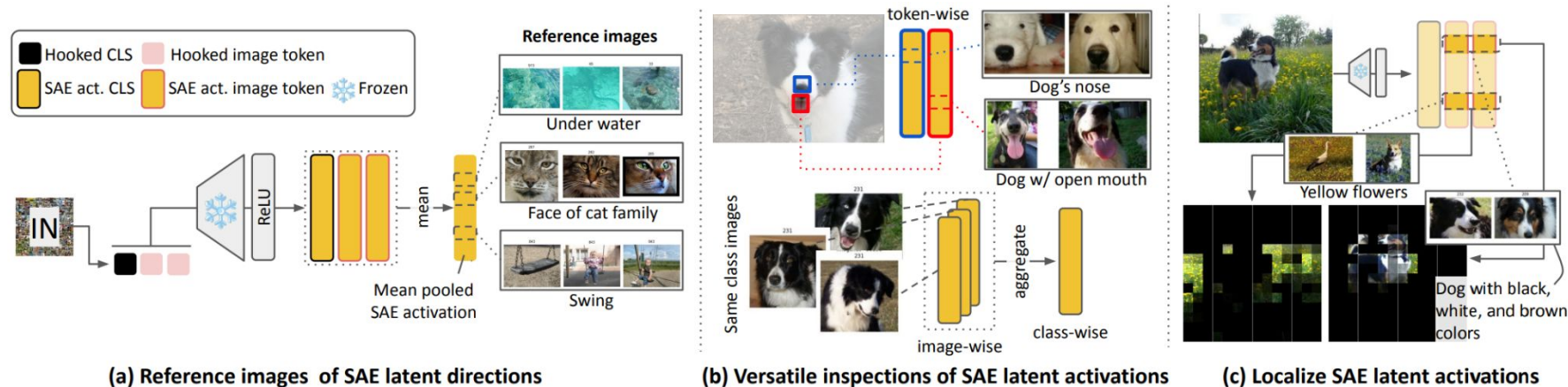


Figure 2: Analyzing SAE latents. (a) We take an average over patch-level activations for an image and keep top- k images having the highest mean activation as the reference images for each SAE latent. (b) From patch-level latent activations, we investigate localized concepts. Furthermore, we represent image-, class-, and dataset-wise concepts by aggregating the patch-level activations. (c) For a certain concept, we provide the spatial attribution of the concept by visualizing the patch-level activations as a segmentation mask.

The effectiveness of PatchSAE

PatchSAE identifies diverse interpretable concepts.

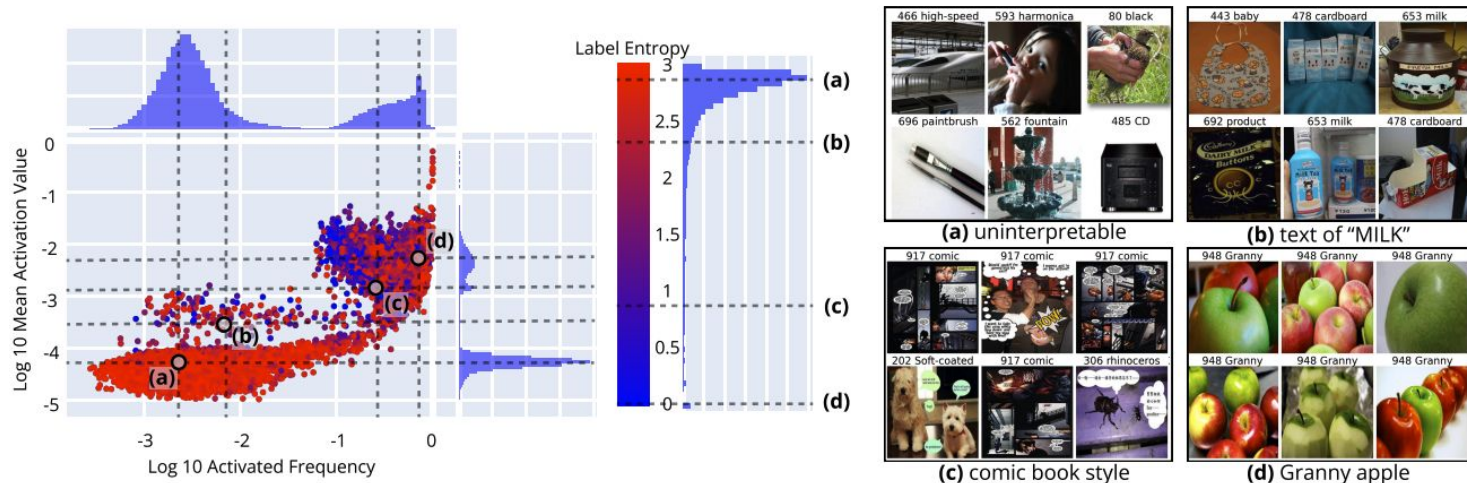


Figure 3: SAE latents statistics and reference images. Left: Scatter plot of SAE latent statistics (x -axis: $\log_{10}$ of activated frequency, y -axis: $\log_{10}$ of mean activation) colored by label entropy. Right: Reference images from Imagenet of four SAE latents in different regions.

The effectiveness of PatchSAE

Latents that activate for specific text appearing in images



Figure 13: SAE latents can be multimodal. We find multimodal SAE latents that activate from both text and images of the same concept. Each row shows different SAE latent. The first column shows the segmentation mask from the input image and the right five columns are reference images. From top to bottom, each latent represents blue, tooth, (cup)cake, mouse (as a representative animation character), good, Christmas, and greetings (hello and welcome). SAE latent activation value and more examples are provided in the [interactive](#)

The effectiveness of PatchSAE

PatchSAE provides spatial attribution of concepts.

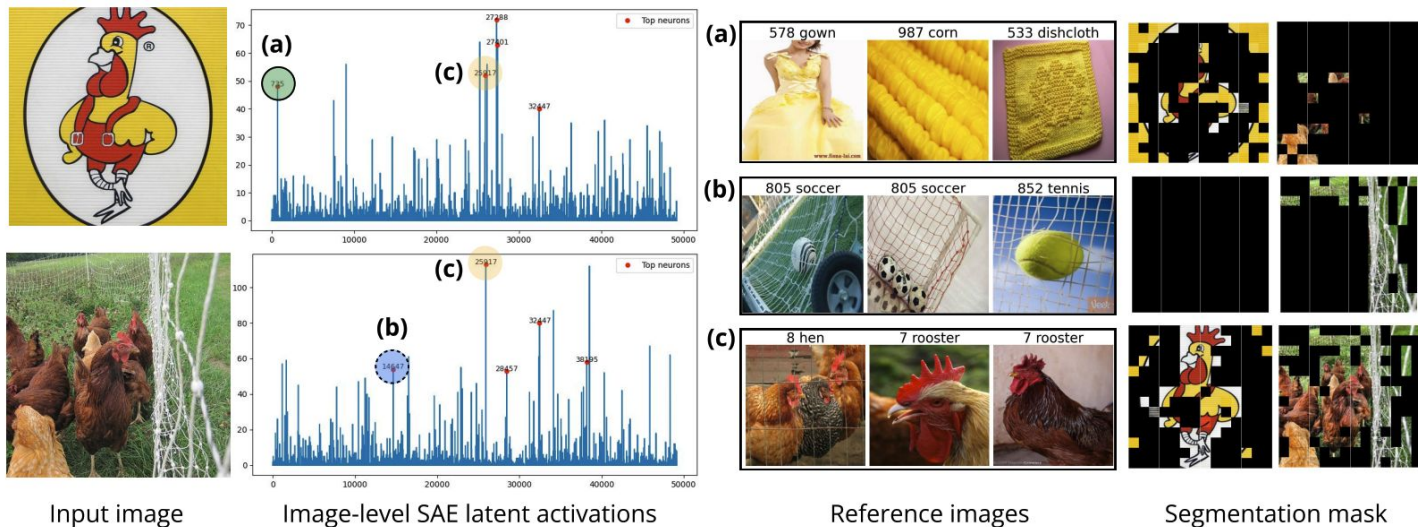


Figure 4: Localizing SAE latent activations under a covariate shift. Given two input images of class hen, we show image-level aggregated SAE latent activations (x -axis: SAE latents index y -axis: image-level activation), reference images from ImageNet, and segmentation masks for each input are shown. Among top 10 latents for each input, we pick three interpretable indices where (a) and (b) represent different domains (image style or background) and (c) shows the shared concept.

The effectiveness of PatchSAE

PatchSAE generalizes across different datasets.

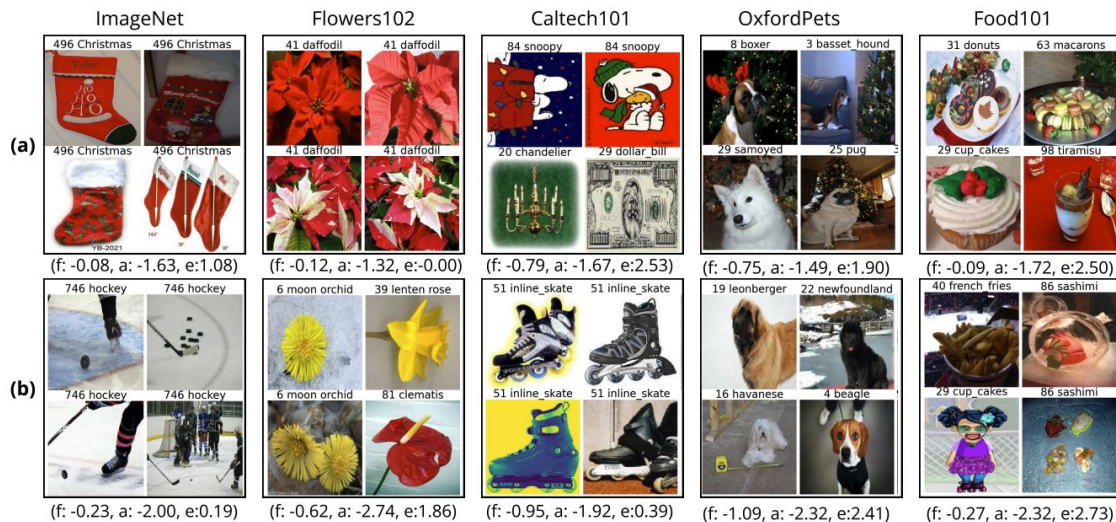
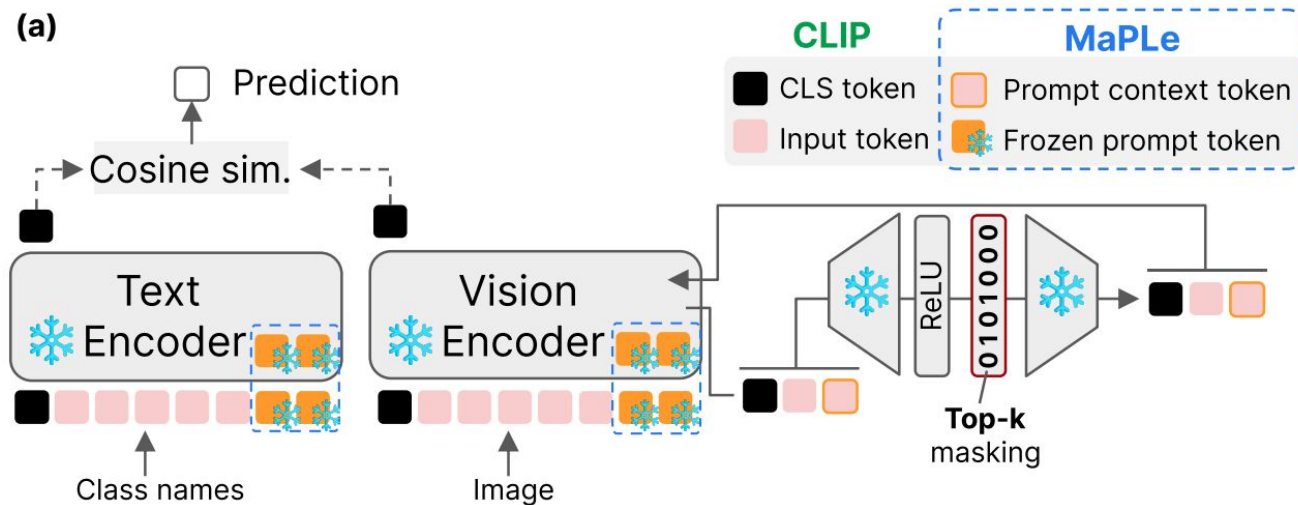


Figure 10: SAE latents are generalizable to different datasets. Reference images of two SAE latents (a) (top) and (b) (bottom) from five datasets. We present label and class name above each image. The latent statistics $\log_{10}$ of activated frequency, $\log_{10}$ of mean activation, and label entropy values computed from each dataset are summarized as (f, a, e) below four reference images. More examples are shown in the [interactive demo](#).

Impact of SAE Latents on Classification

1. CLIP image tokens $\rightarrow$ PatchSAE encoder;
2. Activate sparse latent representations;
3. Decode into reconstructed patch tokens;
4. Feed into the remaining CLIP layers and compute similarity with text embeddings to obtain the classification result.

(a)



Task: ImageNet-1k
classification
Model: CLIP ViT-B/16

Top- k SAE Latent Masking

Aim: To check whether a small number of SAE units can *reconstruct meaningful class-specific representations*.

1. Compute class-level activations:

- For each class (e.g., “dog”), use images from that class in the training split.
- Aggregate their SAE latent activations.
- Identify the top- k most frequently activated latent units for that class → called class-representative latents.

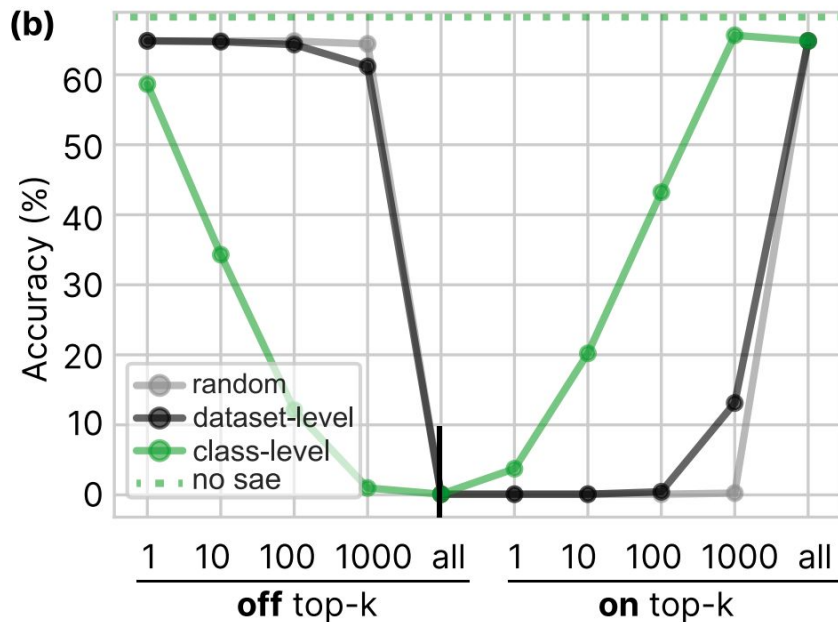
2. Before decoding the SAE output, **mask the latent activation vector**.

- “**on top- k** ”: Only allow the top- k latents for that class to remain active (others zeroed).
- “**off top- k** ”: Keep all latents except the top- k active (the top- k are set to zero).

3. Take the reconstructed output of the SAE (from masked latents).

4. Use this in place of CLIP’s original vision encoder output.

Results for the top- k SAE latent masking



Ablating a small number of class-specific latents severely degrades accuracy.

- Removing just the top 10 class-level latents drops accuracy significantly.
- In contrast, removing random or general dataset-level latents has little impact until many are removed.

PatchSAE successfully identifies class-discriminative concepts that are crucial for classification

Understanding Adaptation Mechanism

Question:

- Does adaptation make models more prominently activate class discriminative concepts?
- Do they define new mappings between the used concepts and the downstream task classes?

Method: Compare class-discriminative SAE latents before (original CLIP) and after adaptation. Use the same PatchSAE (trained on CLIP) to analyze both models.

Analysis: For each class, compare the set of top-activating SAE latents between CLIP and MaPLe.

Understanding Adaptation Mechanism: Evaluation Setup

Base-to-novel classification: Classes from each dataset are split into two groups

- The first half as **Base** classes (seen)
- The second half as **Novel** classes (unseen)

MaPLe:

- Adds learnable prompt tokens to frozen CLIP
- Injects prompts into input and early layers of both text and image encoders
- Few-shot adaptation: Trained on few-shot samples from **base classes only**

Transferring SAEs to Adapted Models

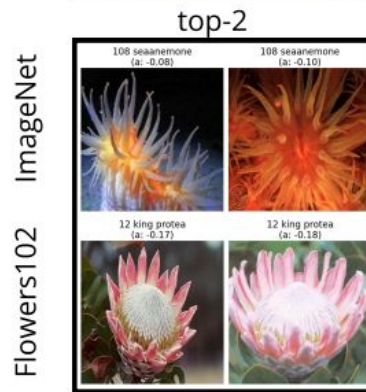
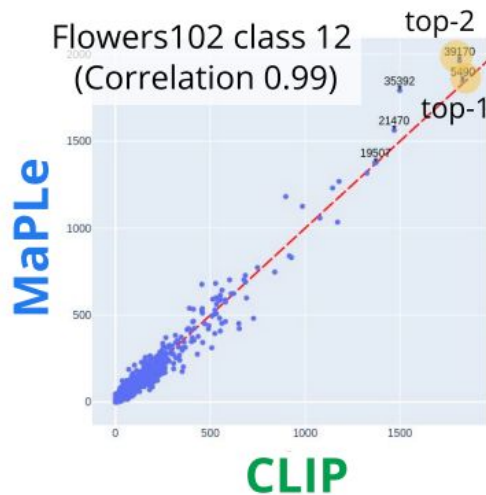
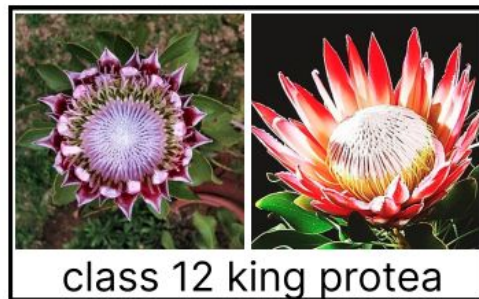
Method:

- Apply the same PatchSAE (trained on CLIP) to both CLIP and MaPLE.
- Compare model behavior in the shared SAE latent space to probe how adaptation affects internal representations.
- Repeat top-k SAE latent masking under various backbone configurations:
 - PatchSAE training backbone: CLIP
 - Latent activation backbone: CLIP or MaPLE
 - Inference backbone: CLIP or MaPLE

Why?

- Prompt-based adaptation keeps the original model frozen and only add learned prompt tokens -> Theoretically, MaPLE's internal representation space should be close to original CLIP.
- **If an SAE can only interpret CLIP but completely fails to work on MaPLE**
 - Its representation capacity is overfitted
 - MaPLE has altered the model's internal representation space.
- **If the SAE also performs well on MaPLE**
 - The SAE has learned general, stable semantic directions in the model
 - The adaptation does not disrupt the model's internal structure but merely guides it to focus on different directions.

Comparing Top SAE Latents Across Models



Reference image

Concept Reweighting after Adaptation

High-High: High activation in both CLIP and MaPLE -> stable, shared, important concepts

High-to-Low: Strong in CLIP, reduced in MaPLE -> De-emphasized after adaptation

Low-to-High: Weak in CLIP, strong in MaPLE -> Newly emphasized after adaptation

Conclusion:

- Adaptation $\neq$ Learning new features
- Adaptation = Reusing features more strategically

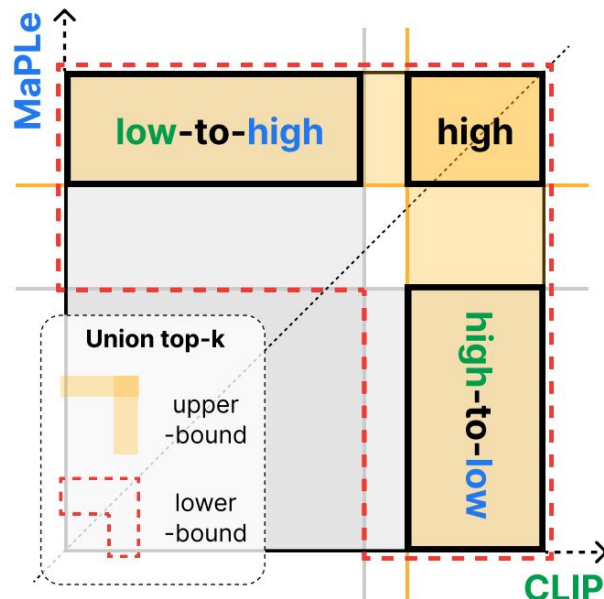


Figure 7: Scatter plot regions (see Fig. 17).

Finding 1: Overlap in Top Activating SAE Latents

Most top activating SAE latents are shared between CLIP and MaPLe.

Table 1: Comparison on class-level SAE latents. The first six columns show the average number of class-level latents in three groups: high, high-to-low, low-to-high groups per class. Details about three groups are provided in Figure 17. Gray colored for values below 0.1. The next four columns show the accuracy (the same results as dashed lines in Fig. 5(c)) and the final last column shows the performance improvement rate Δ (§ 4.2.1). Rows are sorted by Δ .

Dataset	SAE latents count (average)						Accuracy (%)				$\Delta(\%) \uparrow$
	high		high-to-low		low-to-high		CLIP		MaPLe		
	base	novel	base	novel	base	novel	base	novel	base	novel	
StanfordCar	11.18	11.10	0.00	0.12	0.12	0.16	53.45	66.46	56.16	60.63	5.82
FGVC Aircraft	10.10	10.57	0.31	0.32	0.00	0.03	21.72	28.09	31.19	30.07	12.10
ImageNet	11.05	10.82	0.02	0.02	0.12	0.22	69.88	67.23	73.80	68.24	13.01
Food101	11.24	11.58	0.00	0.00	0.00	0.00	84.85	86.65	87.26	89.14	15.91
SUN397	11.20	11.31	0.00	0.01	0.02	0.02	68.74	72.27	77.89	76.26	29.27
UCF101	10.90	11.10	0.01	0.01	0.07	0.06	66.56	68.80	81.13	71.40	43.57
OxfordPet	11.11	11.73	0.00	0.00	0.24	0.16	85.41	95.26	92.04	94.99	45.44
DTD	8.85	8.96	0.32	0.36	1.68	2.02	53.12	53.44	75.17	55.62	47.03
EuroSAT	6.90	8.70	0.50	1.00	4.10	2.30	42.61	62.42	73.07	55.77	53.08
Caltech101	11.25	11.32	0.02	0.00	0.00	0.00	92.57	93.47	97.49	94.29	66.22
Flowers102	10.81	11.47	0.01	0.00	0.01	0.01	56.32	69.26	88.07	68.41	72.69

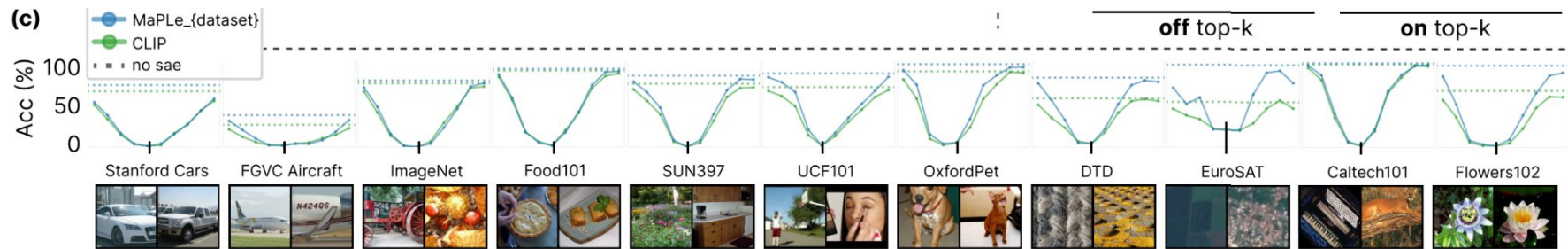
Finding 2: Influence of top SAE latents in CLIP and MaPLE are different

Question: If CLIP and MaPLE activate the same concepts, why does MaPLE perform better?

Experiment: Repeat the "top-k masking" experiment, but now compare the performance of CLIP and MaPLE when using the exact same set of SAE latents.

Hypothesis: Even with the same underlying visual concepts, the adapted model (MaPLE) has learned to interpret or weigh them differently to make better predictions.

Evidence 1: MaPLe is better at using the concepts



Given the **same number of active SAE latents**, MaPLe consistently achieves higher accuracy than CLIP.

The performance gap is not about what concepts are available, but about how effectively the model uses them -> MaPLe makes better use of the same information.

Evidence 2: Case Studies (Flowers102 and EuroSAT)

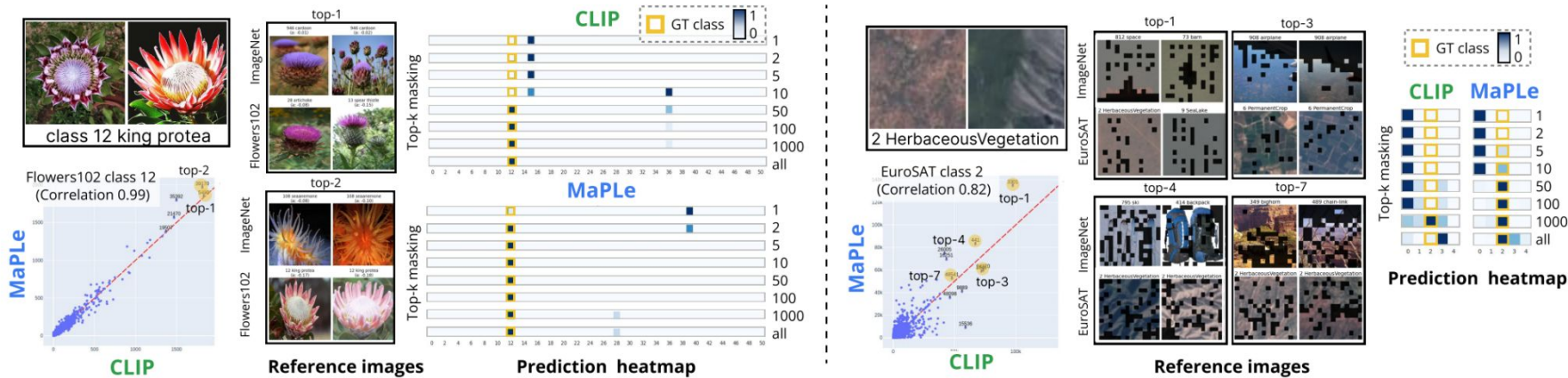


Figure 6: Different impact of SAE latents. We show two example cases of Flowers102 (left) and EuroSAT (right). In each figure, we show two example images of the ground-truth (GT) class, a scatter plot view of class-level SAE latent activations comparison (x -axis: CLIP, y -axis: MaPLE trained on the base classes of the downstream task), reference images from ImageNet and the downstream dataset for top latents, and the top- k masking results as a prediction heatmap. We show reference images with segmentation masks for EuroSAT.

Takeaways

- PatchSAE is an effective framework for extracting spatially-attributed, interpretable concepts from ViT.
- Provides a concrete way to "debug" and analyze a model's behavior on the level of concepts.
- The performance gains from prompt-based adaptation (e.g., MaPLe) do not primarily come from learning new visual features.
- Instead, adaptation is a process of selective remapping: it learns to assign the correct importance to existing that are already present in the base model.
- The base model already knows what's needed, and adaptation teaches it how to use that knowledge for the new task.